NATIONAL UNIVERSITY OF TECHNOLOGY

SMART HOSTEL MANAGEMENT SYSTEM

Database-Driven Web Application

Field	Details
Project Title	Smart Hostel Management System
Subject	Database Systems / problem-based learning (PBL)
Technology Stack	PHP 8.0 | MySQL (MariaDB 10.4) | HTML5 | CSS3 | JavaScript
Database Tool	phpMyAdmin 5.2.1
Server	Apache (XAMPP / localhost)
Academic Level	University Undergraduate
Submission Date	May 2026

Abstract

The Smart Hostel Management System is a web-based database management system developed to simplify and improve the day-to-day administrative operations of a university hostel. Traditional hostel management relies heavily on paper-based processes and manual record-

keeping, which are both time-consuming and error-prone. This project solves these problems by creating a centralized digital system using PHP and MySQL.

The system manages thirteen interconnected database entities: students, rooms, staff, complaints, room allocations, room transfers, invoices, payments, mess records, entry logs, leave requests, visitors, and visitor logs. The front-end interface is built using HTML5, CSS3, and JavaScript, offering a modern, responsive dashboard experience. Core operations including adding, searching, and deleting records are working properly through form-based interactions backed by PHP server-side logic.

The project helped us gain practical experience in relational database design, entity-relationship modeling, SQL query formulation, and full-stack web development. The final result is a functional administrative panel that an institution can deploy to replace manual hostel record management.

Introduction

A hostel is more than just a place to sleep — it is a daily living environment for hundreds of university students. Managing a hostel involves tracking student admissions, room assignments, fee collection, mess attendance, complaints, visitor entries, and leave requests. When done manually, even small hostels struggle to maintain accurate and up-to-date records.

The Smart Hostel Management System was developed as a problem-based learning (PBL) project to provide a practical, technology-based solution to these challenges. It combines relational database concepts, server-side programming, and front-end interface design into one single system.

The system is built on the LAMP-equivalent stack: Apache web server, MySQL (MariaDB) for the database, and PHP for server-side logic, with HTML/CSS/JavaScript powering the browser interface. All data persisted in a normalized relational database named hostel_management, which contains thirteen carefully designed tables connected through foreign key relationships.

From the moment a student is admitted to the hostel, all information — room allocation, monthly invoices, mess meals, entry and exit logs, complaints, and visitor visits — is recorded in the system. Staff members can add, search, view, and delete records from a single unified dashboard accessible through a web browser.

Problem Statement

University hostels across many institutions continue to rely on manual, paper-based record systems. This approach creates several common problems:

- Data redundancy and inconsistency: The same student information is often recorded across multiple registers, leading to conflicting data.
- Slow retrieval: Finding a student record, room allocation, or complaint status requires physical searching through files.
- Human error: Manual entry of fees, room numbers, and attendance is highly susceptible to mistakes.
- No audit trail: There is no reliable way to track who entered which data or when changes were made.
- Difficulty generating reports: Summarizing monthly fees, occupancy rates, or pending complaints requires significant effort.
- Visitor and gate security: Manual visitor logs are unreliable and easily tampered with.
- Lack of complaint tracking: Complaints raised by students often go unresolved because there is no formal tracking mechanism.

These problems collectively reduce the efficiency of hostel administration and negatively affect the student experience. A digital system that centralizes all records in a structured relational database is the appropriate solution.

Objectives

The main objectives of the Smart Hostel Management System are:

1. To design and implement a normalized relational database that stores all hostel-related records without data redundancy.
2. Building a user-friendly web interface that allows hostel staff to perform CRUD (Create, Read, Update, Delete) operations on all records.
3. To enable real-time dashboard statistics showing total students, rooms, staff, and complaints.
4. To implement a record search feature that retrieves full details of any student, staff member, room, or complaint using their unique ID.
5. To support room allocation and room transfer management, tracking which student lives in which room.
6. To automate invoice generation linking student IDs with monthly rent, mess charges, and penalties.
7. To track student entry and exit from the hostel premises with late status logging.
8. To manage visitor records and log visitor entry and exit times with CNIC verification.
9. To provide a complaint submission and status tracking module for hostel residents.
10. To demonstrate practical application of SQL joins, foreign keys, constraints, and normalization concepts.

Scope of Project

In Scope

- Student registration, profile management, and deletion.
- Room creation, detail viewing, and deletion.
- Staff member registration and profile management.
- Complaint logging and status tracking (Pending / Resolved).
- Room allocation — assigning students to rooms.
- Room transfer requests and staff approval.
- Monthly invoice generation covering rent, mess, and penalties.
- Payment recording against invoices.
- Mess meal attendance tracking per student per day.
- Entry and exit log recording with late status.
- Leave requested submission and status tracking.
- Visitor registration with CNIC and relation type.
- Visitor entry and exit time logging.
- Real-time statistics dashboard.
- Search / look up any record by primary key.

Out of Scope

- Student-facing self-service portal (students cannot log in directly).
- Mobile application interface.
- Email or SMS notification system.
- Automated invoice generation via scheduled jobs.
- Integration with university ERP or payment gateways.
- Role-based access control (admin vs. warden vs. manager roles).

Technologies Used

Backend

Technology	Version	Purpose
PHP	8.0.30	Server-side scripting, form handling, database queries
MySQL (MariaDB)	10.4.32	Relational database management system
Apache HTTP Server	XAMPP	Web server hosting PHP files on localhost
phpMyAdmin	5.2.1	GUI tool for database creation and management

Frontend

Technology	Purpose
HTML5	Page structure and semantic markup
CSS3 (custom)	Styling, layouts (grid/flexbox), responsive design, animations
JavaScript (vanilla)	Modal interactions, search overlay, navbar scroll effect, toast notifications
Google Fonts (Bebas Neue, DM Sans)	Typography — display headings and body text

Development Environment

Tool	Role
XAMPP	Localhost stack: Apache + MySQL + PHP
phpMyAdmin	SQL dump generation and database inspection
Web Browser	Application testing and UI verification
Text Editor / IDE	Code authoring

System Requirements

Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 / AMD equivalent (1.6 GHz+)
RAM	4 GB (8 GB recommended)
Storage	500 MB free disk space
Network	Not required (localhost); LAN for multi-user deployment
Display	1280 x 720 resolution minimum

Software Requirements

Software	Requirement
Operating System	Windows 7+ / Ubuntu 18+ / macOS 10.14+
Web Server	Apache 2.4+ (included in XAMPP)
PHP	Version 8.0 or higher
Database	MySQL 5.7+ / MariaDB 10.4+
Browser	Chrome 90+, Firefox 85+, Edge 90+, Safari 14+
phpMyAdmin	Version 5.x (for SQL import)

Methodology

The project was completed in multiple development phases with the following phases:

Phase 1 — Requirements Analysis

We identified the core administrative operations of a university hostel. This included understanding the data entities (students, rooms, staff), relationships between them (a student occupies a room, a staff member approves a transfer), and the operations required (CRUD, search, reporting).

Phase 2 — Database Design

An Entity-Relationship (ER) diagram was drawn to identify entities, their attributes, and relationships. This was then translated into a normalized relational schema. The database schema was finalized with thirteen tables, primary keys, and foreign key constraints enforced at the database level using InnoDB engine.

Phase 3 — Database Implementation

The SQL script hostel_management.sql was written and executed in phpMyAdmin. Tables were created with appropriate data types, AUTO_INCREMENT primary keys, and referential integrity constraints. The script was verified to generate the correct schema without errors.

Phase 4 — Backend Development

PHP scripts were written to connect to the database using mysqli_connect(). Server-side logic was developed for all CRUD operations: adding students, rooms, staff, and complaints; deleting records (with cascading deletes for student records); and searching by primary key with JOIN queries for relational lookups.

Phase 5 — Frontend Development

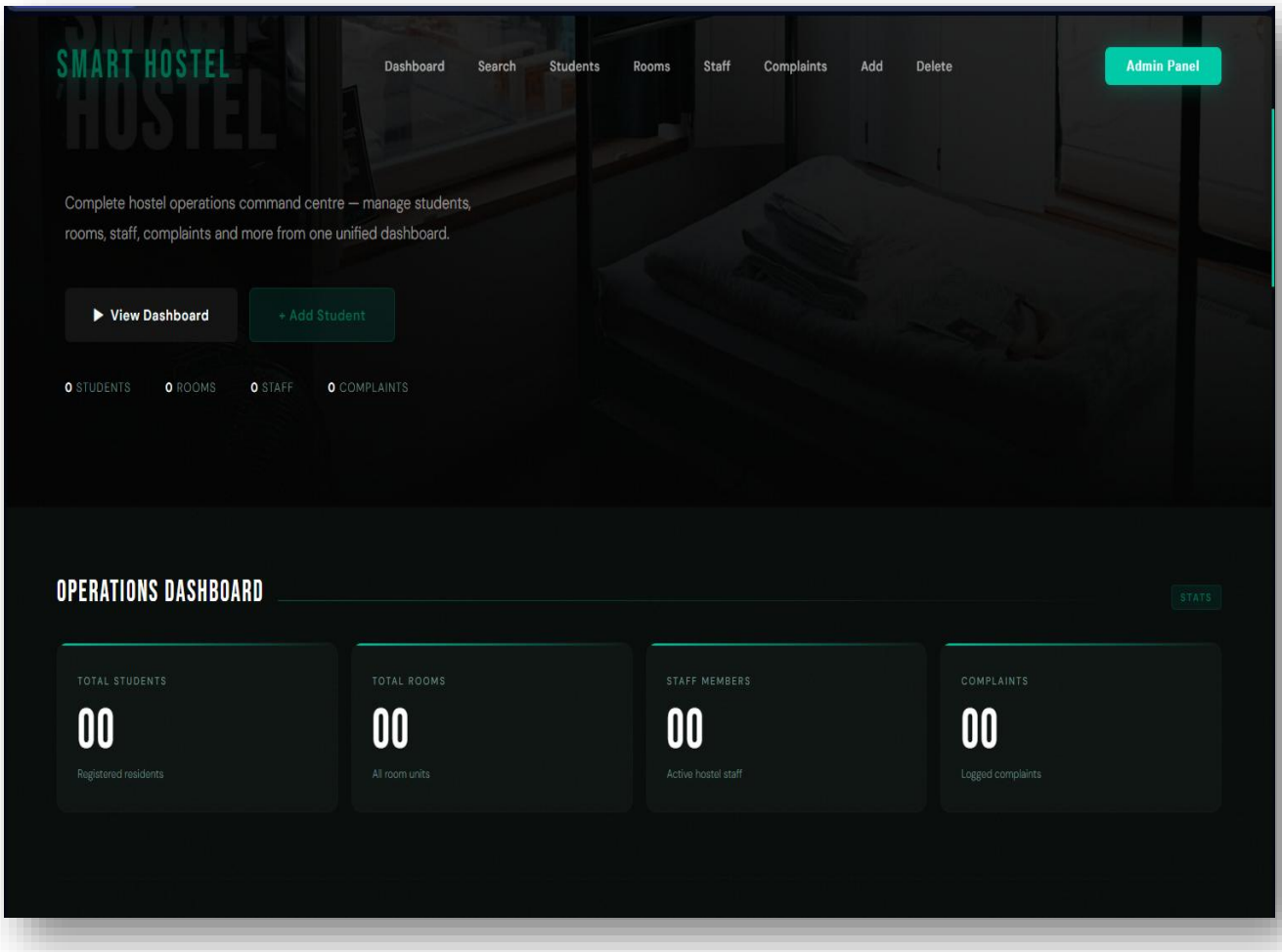
The HTML/CSS/JavaScript interface was designed to be easy to use and visually organized. A dark-themed dashboard with a teal color scheme was developed. The layout uses CSS Grid and Flexbox. JavaScript was added for modal search popups, toast notifications, and navbar scroll behavior. The design is responsive for mobile screens.

Phase 6 — Testing

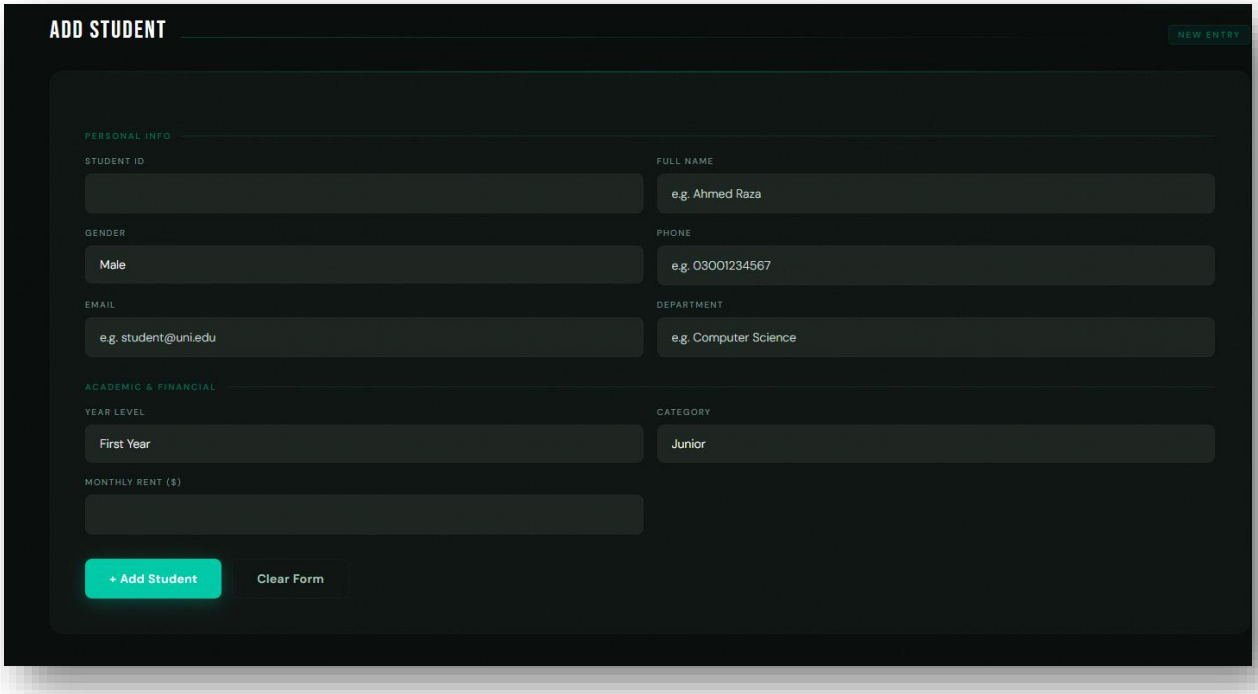
The system was tested by inserting sample records and verifying that all forms submitted correctly, search results displayed accurate data, deleted operations cascaded properly, and the dashboard statistics reflected live database counts.

Screen Shots

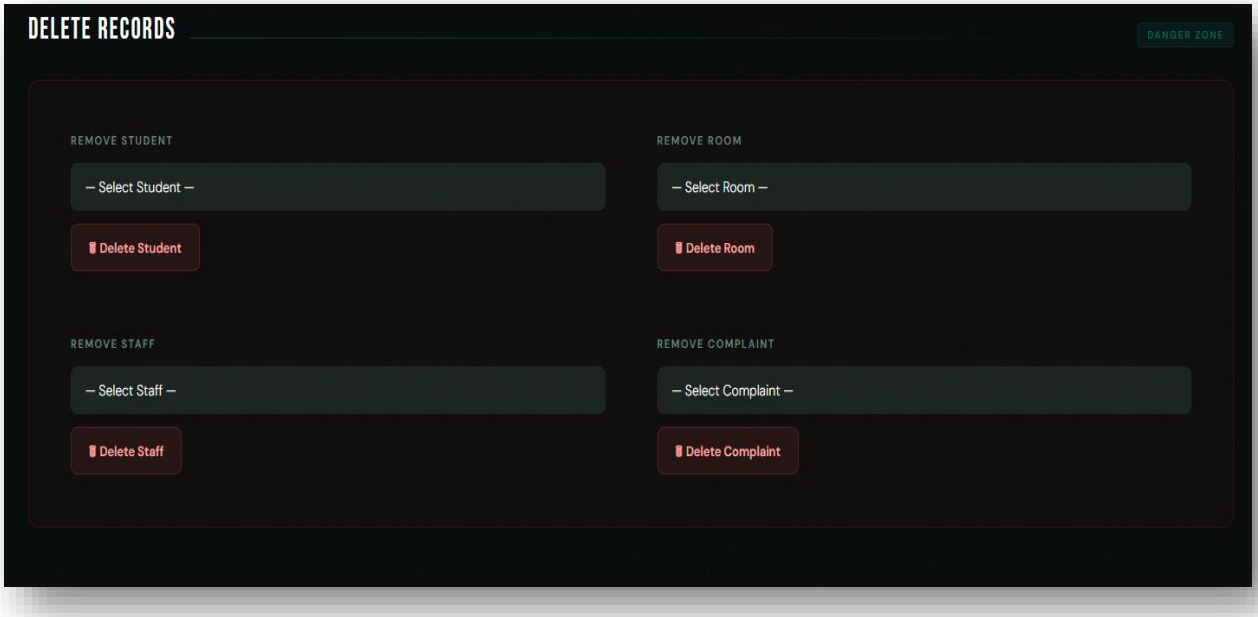
Dash Board



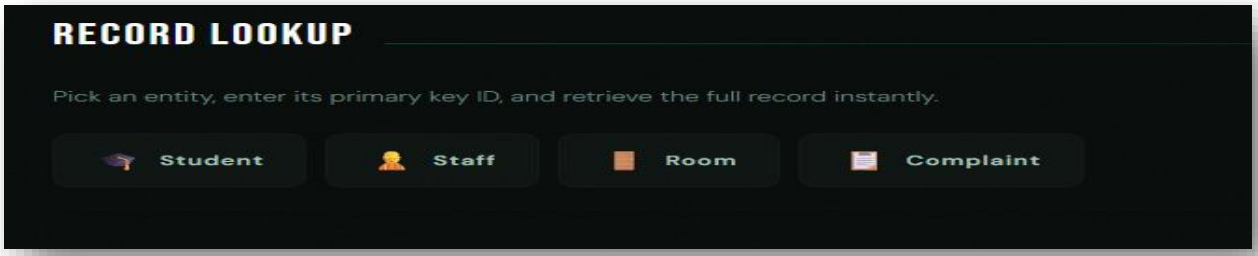
Add Record



Delete records



Search Records



Database Design

The database hostel_management is developed on MariaDB 10.4 using the InnoDB storage engine, which supports foreign key constraints and ACID-compliant transactions. The database consists of thirteen tables, each representing a distinct real-world entity in hostel operations.

Normalization

The schema adheres to Third Normal Form (3NF):

- 1NF: All columns contain atomic values. No repeating groups.
- 2NF: Every non-key attribute is working properlyly dependent on the entire primary key (all tables use single-column primary keys).
- 3NF: No transitive dependencies exist. For example, student department is stored in the student table, not duplicated in Room_Allocation or Invoice.

Storage Engine

All tables use the InnoDB engine, which enforces referential integrity through foreign key constraints. This helps make sure that, for example, a complaint cannot reference a student_ID that does not exist in the student table.

Character Encoding

All tables use utf8mb4 with utf8mb4_general_ci collation, providing full Unicode support including multi-language names and special characters.

Key Design Choices

- AUTO_INCREMENT primary keys on all tables ensure unique identification without manual assignment.
- VARCHAR and TEXT types are used for string fields, with appropriate lengths to prevent storage waste.
- DECIMAL(10,2) is used for all monetary values (rent, fees, amounts) to ensure precision in financial calculations.
- DATETIME is used for timestamp fields (Created_At, Entry_Time, Exit_Time) to record exact moments.

- DATE is used where only the calendar date matters (Meal_Date, Payment_Date, Start_Date, End_Date).
- Status fields use VARCHAR(20) or VARCHAR(30) and store values like 'Pending', 'Approved', and ‘Resolved'.

Entity-Relationship (ER) Diagram Explanation

The ER diagram (provided in the pdf files/erd diagram.pdf and word files/er diagram.docx included in the project submission) shows all thirteen entities and their relationships. The following explains the key relationships:

Core Entities

Student (Central Entity)

Student is the most central entity in the system. It directly or indirectly connects to ten other tables. A student has one primary key (Student_ID) and participates in relationships with Room_Allocation, Complaint, Invoice, Mess_Record, Entry_Log, Leave_Request, Visitor, and Room_Transfer.

Room (Physical Resource)

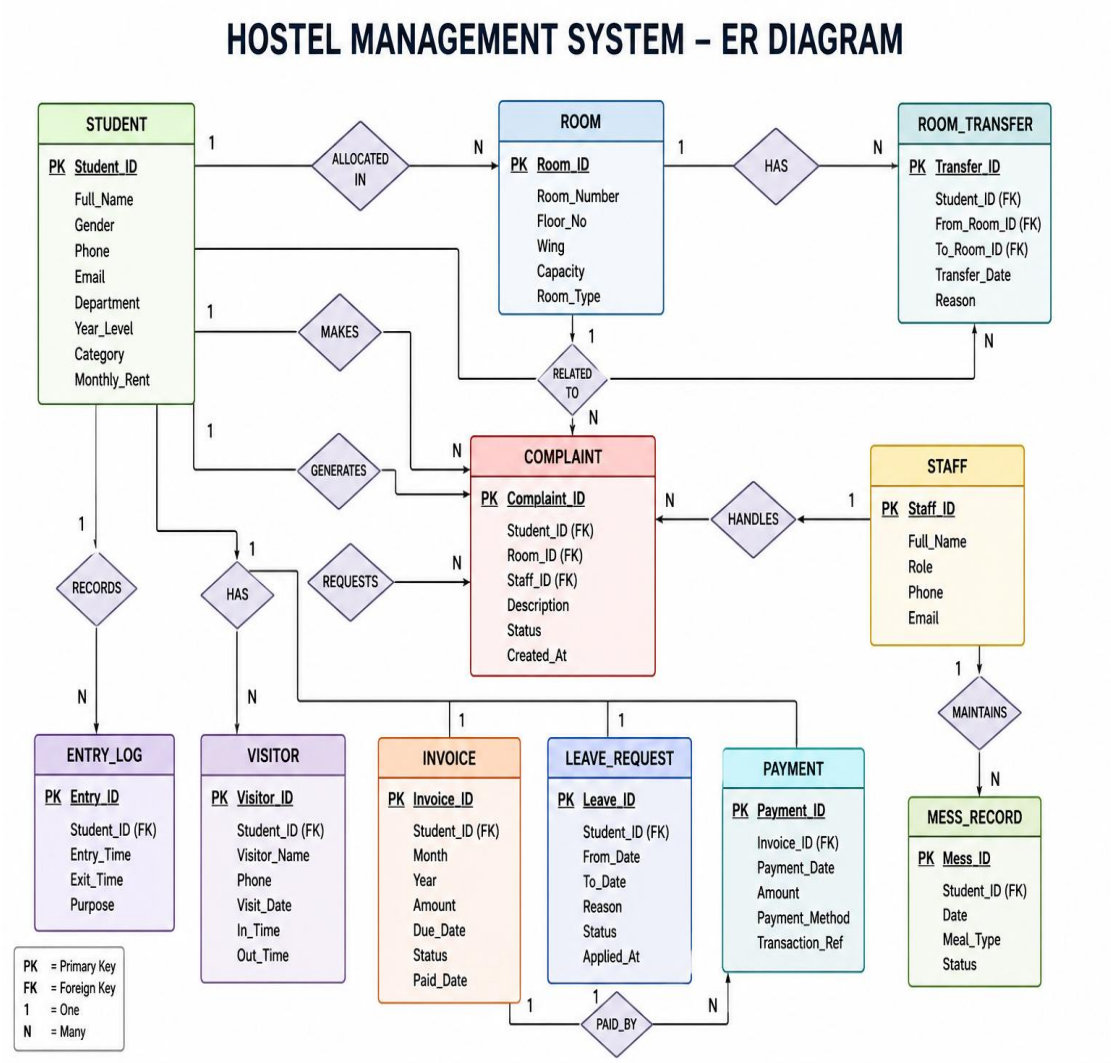
Room represents physical hostel units. It connects to Room_Allocation (which student occupies which room) and Complaint (complaints filed for a specific room). Room_Transfer also references Room twice — once for the current room and once for the requested room.

Staff (Human Resource)

Staff represent hostel employees. It connects to Room_Transfer as the approving authority, establishing an M:1 relationship where one staff member can approve many transfer requests.

Relationship Types

Relationship	Type	Description
Student — Room_Allocation	1:M	One student can have multiple allocation records over time
Room — Room_Allocation	1:M	One room can have multiple allocation records
Student — Complaint	1:M	One student can file multiple complaints
Room — Complaint	1:M	One room can have multiple complaints filed against it
Student — Invoice	1:M	One student receives multiple monthly invoices
Invoice — Payment	1:M	One invoice can be paid via multiple partial payments
Student — Mess_Record	1:M	One student has multiple meal records
Student — Entry_Log	1:M	One student has multiple entry/exit log entries
Student — Leave_Request	1:M	One student can submit multiple leave requests
Student — Visitor	1:M	One student can have multiple registered visitors
Visitor — Visitor_Log	1:M	One visitor can have multiple log entries across visits
Student — Room_Transfer	1:M	One student can request multiple room transfers
Staff — Room_Transfer	1:M	One staff member can approve multiple transfers


HOSTEL MANAGEMENT SYSTEM – ER DIAGRAM
STUDENT
PK Student_ID
Full_Name
Gender
Phone
Email
Department
Year_Level
Category
Monthly_Rent
ALLOCATED IN
ROOM
PK Room_ID
Room_Number
Floor_No
Wing
Capacity
Room_Type
HAS
ROOM_TRANSFER
PK Transfer_ID
Student_ID (FK)
From_Room_ID (FK)
To_Room_ID (FK)
Transfer_Date
Reason
MAKES
RELATED TO
GENERATES
COMPLAINT
PK Complaint_ID
Student_ID (FK)
Room_ID (FK)
Staff_ID (FK)
Description
Status
Created_At
HANDLES
STAFF
PK Staff_ID
Full_Name
Role
Phone
Email
RECORDS
HAS
REQUESTS
MAINTAINS
ENTRY_LOG
PK Entry_ID
Student_ID (FK)
Entry_Time
Exit_Time
Purpose
VISITOR
PK Visitor_ID
Student_ID (FK)
Visitor_Name
Phone
Visit_Date
In_Time
Out_Time
INVOICE
PK Invoice_ID
Student_ID (FK)
Month
Year
Amount
Due_Date
Status
Paid_Date
LEAVE_REQUEST
PK Leave_ID
Student_ID (FK)
From_Date
To_Date
Reason
Status
Applied_At
PAYMENT
PK Payment_ID
Invoice_ID (FK)
Payment_Date
Amount
Payment_Method
Transaction_Ref
PAID_BY
MESS_RECORD
PK Mess_ID
Student_ID (FK)
Date
Meal_Type
Status
PK = Primary Key
FK = Foreign Key
1 = One
N = Many

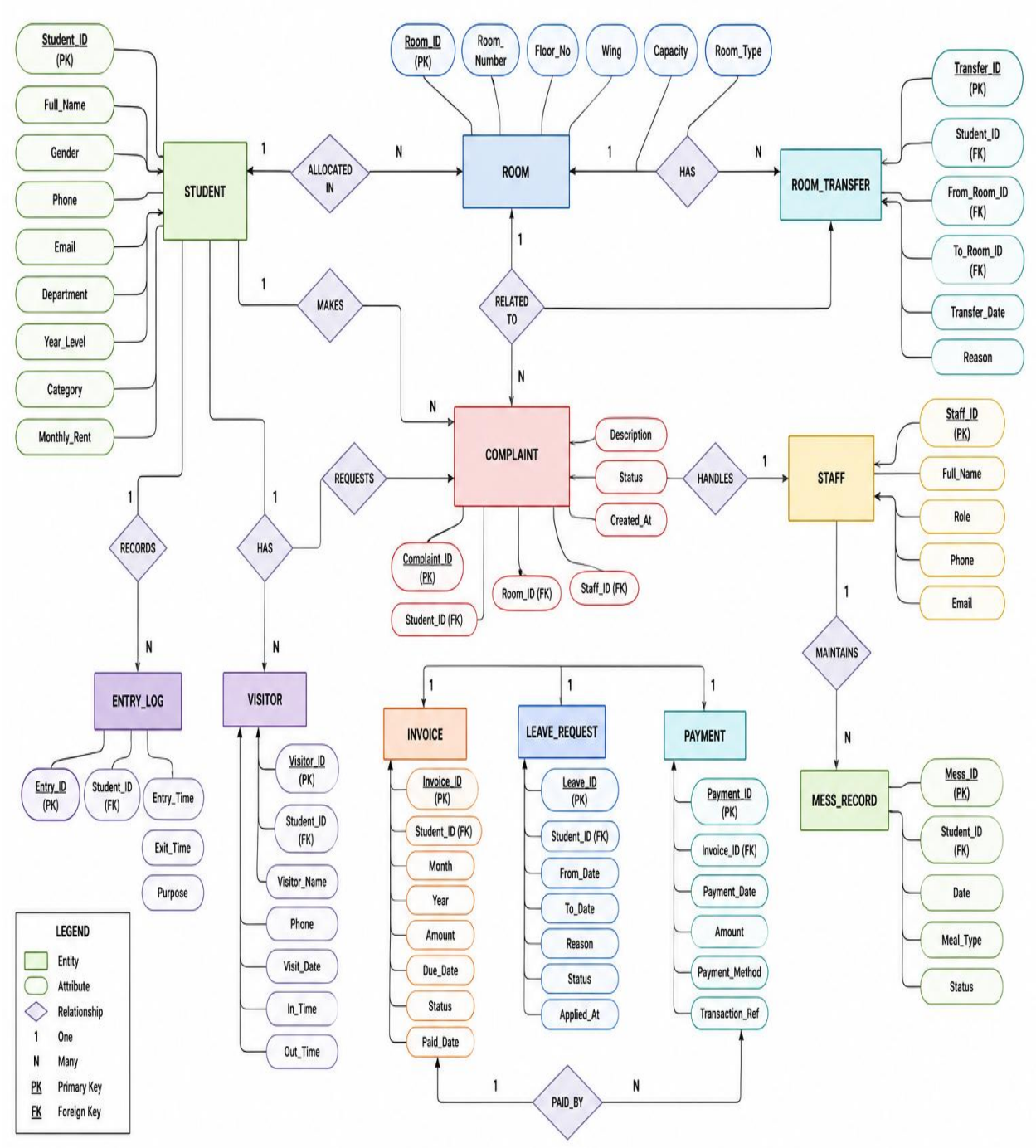

Student_ID (PK)
Full_Name
Gender
Phone
Email
Department
Year_Level
Category
Monthly_Rent
STUDENT
ALLOCATED IN
ROOM
Room_ID (PK)
Room_ Number
Floor_No
Wing
Capacity
Room_Type
HAS
ROOM_TRANSFER
Transfer_ID (PK)
Student_ID (FK)
From_Room_ID (FK)
To_Room_ID (FK)
Transfer_Date
Reason
MAKES
RELATED TO
COMPLAINT
Description
Status
Created_At
Complaint_ID (PK)
Room_ID (FK)
Staff_ID (FK)
Student_ID (FK)
REQUESTS
HANDLES
STAFF
Staff_ID (PK)
Full_Name
Role
Phone
Email
RECORDS
HAS
MAINTAINS
ENTRY_LOG
Entry_ID (PK)
Student_ID (FK)
Entry_Time
Exit_Time
Purpose
VISITOR
Visitor_ID (PK)
Student_ID (FK)
Visitor_Name
Phone
Visit_Date
In_Time
Out_Time
INVOICE
Invoice_ID (PK)
Student_ID (FK)
Month
Year
Amount
Due_Date
Status
Paid_Date
LEAVE_REQUEST
Leave_ID (PK)
Student_ID (FK)
From_Date
To_Date
Reason
Status
Applied_At
PAYMENT
Payment_ID (PK)
Invoice_ID (FK)
Payment_Date
Amount
Payment_Method
Transaction_Ref
PAID_BY
MESS_RECORD
Mess_ID (PK)
Student_ID (FK)
Date
Meal_Type
Status
LEGEND
Entity
Attribute
Relationship
1 One
N Many
PK Primary Key
FK Foreign Key

Table Descriptions

1. student

The student table is the primary entity of the entire system. It stores the personal, academic, and financial profile of every hostel resident.

Column	Type	Description
Student_ID (PK)	INT AUTO_INCREMENT	Unique identifier for each student
Full_Name	VARCHAR(100)	Student's complete name
Gender	VARCHAR(10)	Male or Female
Phone	VARCHAR(20)	Contact phone number
Email	VARCHAR(100)	Email address
Department	VARCHAR(100)	University department (e.g., Computer Science)
Year_Level	VARCHAR(20)	Academic year (First Year to Final Year)
Category	VARCHAR(30)	Student category: Junior, Senior, or Foreign
Monthly_Rent	DECIMAL(10,2)	Fixed monthly rent amount assigned to the student

2. room

Stores details of all physical room units available in the hostel.

Column	Type	Description
Room_ID (PK)	INT AUTO_INCREMENT	Unique room identifier
Room_Number	VARCHAR(20) NOT NULL	Human-readable room label (e.g., 204-A)
Floor_No	INT	Floor number the room is located on
Wing	VARCHAR(30)	Wing of the building (Left or Right)
Capacity	INT	Maximum number of beds in the room
Room_Type	VARCHAR(50)	Shared or Private

3. staff

Stores details of all hostel staff members including their role and contact information.

Column	Type	Description
Staff_ID (PK)	INT AUTO_INCREMENT	Unique staff identifier
Full_Name	VARCHAR(100)	Staff member's full name
Role	VARCHAR(50)	Job role: Warden, Mess Manager, Security Guard, Cleaner, Maintenance Staff
Phone	VARCHAR(20)	Contact phone number
Email	VARCHAR(100)	Email address

4. room_allocation

Tracks which student is assigned to which room and when the allocation was made.

Column	Type	Description
Allocation_ID (PK)	INT AUTO_INCREMENT	Unique allocation record ID
Student_ID (FK)	INT	References student.Student_ID
Room_ID (FK)	INT	References room.Room_ID
Allocation_Date	DATE	Date the student was allocated the room
Status	VARCHAR(20)	Current status (Active, Vacated, etc.)

5. room_transfer

Records requests from students to change their room and the approval status by a staff member.

Column	Type	Description
Transfer_ID (PK)	INT AUTO_INCREMENT	Unique transfer request ID
Student_ID (FK)	INT	References student.Student_ID
Current_Room_ID (FK)	INT	References room.Room_ID — current room
Requested_Room_ID (FK)	INT	References room.Room_ID — desired new room
Reason	TEXT	Student's reason for requesting a transfer
Approval_Status	VARCHAR(20)	Pending, Approved, or Rejected
Approved_By (FK)	INT	Staff References.Staff_ID

6. complaint

Logs of complaints filed by students about their rooms or hostel conditions.

Column	Type	Description
Complaint_ID (PK)	INT AUTO_INCREMENT	Unique complaint ID
Student_ID (FK)	INT	References student.Student_ID
Room_ID (FK)	INT	References room.Room_ID
Description	TEXT	Full description of the complaint
Status	VARCHAR(30)	Pending or resolved
Created_At	DATETIME	Date and time complaint was filed
Resolved_At	DATETIME	Date and time complaint was resolved (nullable)

7. invoice

Generates monthly billing records for each student combining rent, mess charges, and any penalties.

Column	Type	Description
Invoice_ID (PK)	INT AUTO_INCREMENT	Unique invoice ID
Student_ID (FK)	INT	References student.Student_ID
Invoice_Month	VARCHAR(20)	Billing month (e.g., May 2026)
Rent_Amount	DECIMAL(10,2)	Monthly room rent
Mess_Amount	DECIMAL(10,2)	Total mess charges for the month
Penalty	DECIMAL(10,2)	Any additional penalty charges
Total_Amount	DECIMAL(10,2)	Sum of all charges
Status	VARCHAR(20)	Paid or Unpaid

8. payment

Records individual payment transactions made against an invoice.

Column	Type	Description
Payment_ID (PK)	INT AUTO_INCREMENT	Unique payment ID
Invoice_ID (FK)	INT	References invoice.Invoice_ID
Amount_Paid	DECIMAL(10,2)	Amount paid for this transaction
Payment_Date	DATE	Date payment was made
Payment_Method	VARCHAR(30)	Cash, Bank Transfer, Online, etc.

9. mess_record

Tracks daily meal consumption for each student in the hostel mess.

Column	Type	Description
Mess_ID (PK)	INT AUTO_INCREMENT	Unique mess record ID
Student_ID (FK)	INT	References student.Student_ID
Meal_Type	VARCHAR(20)	Breakfast, Lunch, or Dinner
Meal_Date	DATE	Date of the meal
Meal_Cost	DECIMAL(10,2)	Cost charged for that meal

10. entry_log

Records every entry and exit movement of a student at the hostel gate.

Column	Type	Description
Entry_ID (PK)	INT AUTO_INCREMENT	Unique log entry ID
Student_ID (FK)	INT	References student.Student_ID
Entry_Time	DATETIME	Timestamp when student entered the premises
Exit_Time	DATETIME	Timestamp when student exited the premises
Late_Status	VARCHAR(10)	Yes or no — whether the student returned late

11. leave_request

Stores formal leave applications submitted by students when they wish to leave the hostel for a period.

Column	Type	Description
Leave_ID (PK)	INT AUTO_INCREMENT	Unique leave request ID
Student_ID (FK)	INT	References student.Student_ID
Start_Date	DATE	First day of requested leave
End_Date	DATE	Last day of requested leave
Reason	TEXT	Student's reason for the leave
Status	VARCHAR(20)	Pending, Approved, or Rejected

12. visitor

Maintains a registry of individuals who have been cleared to visit students in the hostel.

Column	Type	Description
Visitor_ID (PK)	INT AUTO_INCREMENT	Unique visitor ID
Student_ID (FK)	INT	References student.Student_ID (the student being visited)
Visitor_Name	VARCHAR(100)	Full name of the visitor
CNIC	VARCHAR(30)	National identity card number for verification
Relation_Type	VARCHAR(50)	Relationship with student (Parent, Sibling, Guardian, etc.)

13. visitor_log

Logs each visit event, recording when a registered visitor centers and exits the hostel.

Column	Type	Description
Log_ID (PK)	INT AUTO_INCREMENT	Unique log ID
Visitor_ID (FK)	INT	References visitor.Visitor_ID
Entry_Time	DATETIME	Timestamp of visitor's entry
Exit_Time	DATETIME	Timestamp of visitor's exit

SQL Queries Explanation

The system mainly uses a range of SQL queries for different operations. The following sections explain the key query types used in the project.

1. INSERT Queries — Adding Records

When a user submits the Add Student form, the backend executes an INSERT statement that populates all nine columns of the student table in a single operation. Similar INSERT queries are used for rooms, staff, and complaints. Before inserting, a SELECT query checks duplicate IDs to prevent primary key violations.

2. SELECT Queries — Displaying Lists

The dashboard lists (students, rooms, staff, complaints) use simple SELECT * FROM table ORDER BY column queries. These retrieve all rows from a table and sort them for display. The complaint list is ordered by Complaint_ID DESC to show the most recently logged complaint first.

3. JOIN Queries — Enriched Record Lookup

The search function for complaints uses a multi-table JOIN query. It retrieves the complaint record and simultaneously fetches the student's full name from the student table and the room number from the room table, using LEFT JOIN on the shared foreign key columns. This eliminates the need for multiple separate queries and delivers a complete, human-readable result in one operation.

The room search query uses a LEFT JOIN with COUNT() to calculate how many students are currently allocated to a room, comparing that count against the room's capacity to show occupancy status.

4. DELETE Queries — Removing Records

Deleting a student requires cascading deletes across all dependent tables. The PHP code explicitly deletes related records from eight tables (Room_Allocation, Complaint, Visitor, Mess_Record, Invoice, leave_Request, Entry_Log, Room_Transfer) before deleting the student record itself. This manual cascade is necessary because the database schema uses foreign key constraints that would otherwise prevent deletion of a reference student. Deleting a room similarly clears Room_Allocation records first.

5. COUNT Queries — Dashboard Statistics

The dashboard displays four live statistics: total students, total rooms, total staff, and total complaints. Each is calculated using a SELECT * FROM table query whose result set is passed to mysqli_num_rows() in PHP. This approach counts all rows in each table and displays the result as a real-time metric on the dashboard.

6. Duplicate Check Queries

Before any INSERT operation, a SELECT query with a WHERE clause checks whether a record with the given ID already exists. If mysqli_num_rows() returns greater than zero, the insert is skipped and an error message is shown to the user. This prevents duplicate primary key errors.

7. LIMIT Queries — Single Record Retrieval

Detail pages (student_details.php, room_details.php, staff_details.php, complaint_details.php) use SELECT * FROM table WHERE ID = value LIMIT 1 queries. The LIMIT 1 clause ensures only one row is returned even if a bug were to produce multiple matches, improving performance and predictability.

Modules Description

The system is organized into the following functional modules:

Module 1 — Student Management

- Add new students with personal, academic, and financial details.
- View a grid of all registered students with name, ID, and department.
- Click on any student to view their full profile on a dedicated detail page.
- Delete a student along with all their associated records (cascading delete).
- Search for a student by ID and view their complete profile inline.

Module 2 — Room Management

- Add new rooms specifying room number, floor, wing, capacity, and type.
- View all rooms in a card grid showing key details briefly.
- Click any room to view its full details including current occupancy.
- Delete a room after clearing its allocation records.
- Search for a room by ID and see occupancy vs. capacity.

Module 3 — Staff Management

- Register hostel staff with role, phone, and email.
- View all staff in a card grid.
- Click on any staff to view full profile.
- Delete staff records.
- Search staff by ID.

Module 4 — Complaint Management

- Log complaints by selecting the student and room from dynamic dropdowns.
- All new complaints are automatically assigned Pending status with a Created_At timestamp.
- View complaints in a table with status badges (Pending = amber, Resolved = teal).
- Search for a specific complaint by ID.
- Delete complaint records.

Module 5 — Dashboard & Statistics

- Displays four live count cards: total students, rooms, staff, and complaints.
- Numbers are dynamically fetched from the database on every page load.
- Hero section displays these counts as a quick summary of landing.

Module 6 — Record Lookup (Search)

- Unified search panel allows searching across four entity types.
- A modal popup accepts numeric ID.
- Results are displayed as a structured card with all field values.

- Complaint search uses JOIN queries to show student name and room number.
- Room search shows current occupancy count alongside capacity.

Module 7 — Database-Only Modules

The following modules are fully designed at the database level but not yet wired to a front-end interface. They are part of the schema and represent the system's planned future capabilities:

- Room Allocation — tracks student-room assignments.
- Room Transfer — manages transfer requests and approvals.
- Invoice Generation — monthly billing per student.
- Payment Recording — payments against invoices.
- Mess Record — daily meal tracking.
- Entry Log — gate entry/exit with late status.
- Leave Request — formal leave applications.
- Visitor Registry — CNIC-verified visitor records.
- Visitor Log — visit timestamps.

Frontend Explanation

The front-end of the Smart Hostel Management System is entirely contained in five PHP files that embed HTML, CSS, and JavaScript. The design follows a dark, professional aesthetic with a teal accent color scheme.

Design Theme

The color palette uses a near-black background (#080c0b), with teal (#00c9a7) as the primary accent color, white text for primary content, and muted gray for secondary information. A subtle film grain texture is applied via an SVG filter to give a premium visual texture.

Typography uses two Google Fonts: Bebas Neue for bold display headings (hero title, logo, section headers, stat numbers) and DM Sans for body text and form labels, providing a modern contrast between display and reading type.

Page Layout (index.php)

- Fixed Navbar: Stays at the top while scrolling. Contains the SMART HOSTEL logo, navigation links (smooth-scroll anchors), and an Admin Panel button. The navbar height reduces from 100px to 72px on scroll via JavaScript.
- Hero Section: A full-viewport-height cinematic section with a background image (sourced from Unsplash), overlaid gradient, and a content panel showing the system tagline, call-to-action buttons, and live stats (student/room/staff/complaint counts injected by PHP).
- Dashboard Section: Four stat cards in a CSS Grid layout, each showing a large Bebas Neue number and a label.
- Search Section: Four entity buttons (Student, Staff, Room, Complaint) that open a centered modal overlay.
- Entity Lists: Students, Rooms, and Staff each appear as responsive card grids. Clicking a card navigates to a detail page.
- Complaints Table: A full-width table with status badges (Pending = amber pill, Resolved = teal pill).
- Add Forms: Styled form boxes for adding Students, Rooms, Staff, and Complaints. Forms use a two-column grid layout with labeled inputs, select dropdowns, and success/error message display.
- Delete Zone: A red-tinted danger zone section with four delete forms, each using a dropdown to select the record to remove.
- Footer: Shows the logo and copyright.
- Toast Notifications: Fixed-position slide-up notifications appear after delete operations and auto-dismiss after 3 seconds.

Search Modal

The search modal is a full-screen overlay (backdrop filter: blur) containing a centered card. JavaScript functions openSearch(), closeSearch(), and closeOnBackdrop() handle showing/hiding. The modal accepts a numeric ID and submits to index.php, which processes the search server-side and renders the result below the search section.

Detail Pages

Four separate PHP files serve as detail pages:

- student_details.php: Shows a complete student profile card. Accessed via GET parameter ?id=.
- room_details.php: Shows room number, floor, wing, capacity, and type.
- staff_details.php: Shows staff role, phone, and email.
- complaint_details.php: Shows complaint details with a JOIN query fetching student name and room number.

All detailed pages share the same dark theme styling and include a Back to Dashboard link.

Responsiveness

The layout is responsive via CSS media queries targeting screens narrower than 768px. On mobile, the navbar collapses, hero text scales down, section padding reduces, form grids collapse to single column, and the delete grid goes to single column.

Backend Explanation

The back end is developed entirely in PHP 8.0, using the MySQLi procedural API to interact with the MariaDB database.

Database Connection

Every PHP file begins by establishing a connection using mysqli_connect() with host (localhost), user (root), password (empty, development setting), and database name (hostel_management). Connection failure is managed by die() with an error message.

From Handling Pattern

The back end follows a consistent pattern: when a POST request is received, it checks which submit button was pressed using isset($_POST['button_name']). It then sanitizes input using mysqli_real_escape_string() for string values and intval() for integer values. After sanitization, it checks for duplicates before executing the INSERT query.

Input Sanitization

String inputs are sanitized using mysqli_real_escape_string() to prevent SQL injection via string manipulation. Integer inputs use intval() to ensure they are numeric. HTML output escaped using htmlspecialchars() to prevent XSS attacks. Note: the detail pages (student_details.php, room_details.php, staff_details.php) pass GET parameters directly to queries without intval(), which represents a security vulnerability to address in future versions.

Cascading Deletes

When a student is deleted, the backend loops through eight related tables and executes DELETE WHERE Student_ID = $did on each before deleting the student record. This explicit cascade approach manages referential integrity at the application layer, bypassing the need for ON DELETE CASCADE database constraints.

Query Results Processing

SELECT query results are processed using mysqli_fetch_assoc() in a while loop for lists, and a single call for individual record lookups. Row counts are obtained via mysqli_num_rows(). Boolean query success is checked before rendering results.

Dynamic Dropdowns

The complaint form and delete forms populate their option lists dynamically by running SELECT queries at page load time and echoing PHP-generated option elements. This ensures the dropdowns always reflect the current database state without hardcoding.

Dashboard Statistics

Four COUNT-equivalent queries run on every page load to fetch current totals. The results are stored in PHP variables ($total_students, $total_rooms, $total_staff, $total_complaints) and echoed into the HTML hero section and dashboard cards.

System Workflow

The following steps explain how the system works in practice:

Student Admission Workflow

11. Administrator opens index.php in a browser.
12. Navigates to the Add Student section.
13. Fills in Student ID, name, gender, phone, email, department, year level, category, and monthly rent.
14. Submits the form. PHP checks duplicate ID, then inserts the record.
15. The student now appears in the student's grid on the dashboard.
16. Staff can click the student's name to view their full profile.

Room Assignment Workflow

17. Administrator adds a room via the Add Room form (Room ID, number, floor, wing, capacity, type).
18. A Room_Allocation record is inserted manually or via future UI to assign a student to the room.
19. The room search returns occupancy count by JOINing Room_Allocation with Room.

Complaint Handling Workflow

20. A complaint is logged by selecting the student and room from dropdowns and entering a description.
21. The complaint is inserted with Status = 'Pending' and Created_At = NOW().
22. Staff views the complaints table and identifies pending items.
23. Upon resolution, the status would be manually updated to 'Resolved' (status update UI is a planned enhancement).

Search Workflow

24. Staff click a search entity button (Student, Staff, Room, or Complaint).
25. A modal overlay appears requesting the primary key ID.
26. On submitting, the page reloads and PHP executes the appropriate SELECT (with JOINs for complaint and room).
27. The result is displayed as a structured card below the search section.

Delete Workflow

28. Staff select a record from the Delete Zone dropdown.
29. A browser confirmation dialog appears warning about irreversible deletion.
30. On confirmation, PHP executes cascading deletes across related tables.
31. A toast notification confirms the deletion, and the record disappears from all lists.

Features of the System

- Real-time dashboard with live student, room, staff, and complaint counts.
- Full CRUD (Create, Read, Delete) operations for students, rooms, staff, and complaints.
- Duplicate ID detection before inserting any new record.
- Multi-table JOIN search for complaints (returns student name and room number).
- Occupancy-aware room search showing current occupants vs. capacity.
- Cascading student deletion clearing eight dependent tables automatically.
- Dynamic form dropdowns that auto-populate from live database data.
- Status badge display for complaints (Pending / Resolved) with color coding.
- Individual detail pages for students, rooms, staff, and complaints.
- Animated modal search overlay with keyboard escape and backdrop-click to close.
- Responsive design supporting mobile screen sizes.
- Toast notification system for deleting confirmations.
- Smooth-scroll navigation between dashboard sections.
- Navbar that compresses on scroll for better reading ergonomics.
- Thirteen-table normalized database schema supporting all hostel operations.
- Foreign key constraints enforcing relational integrity at the database level.

Advantages

- Centralized Data Management: All hostel records are stored in one database, eliminating scattered paper files and spreadsheets.
- Speed and Efficiency: Record retrieval that once took minutes in manual systems happens in milliseconds with indexed SQL queries.
- Data Integrity: Foreign key constraints prevent orphan records. Cascading deletes maintain consistency when a student is removed.
- Scalability: The relational scheme supports hundreds of students, rooms, staff, and thousands of log entries without structural changes.
- Reduced Human Error: Form validation and duplicate checks reduce common data entry mistakes.
- Professional UI: The dark-themed, responsive interface provides a modern user experience appropriate for an institutional admin panel.
- Cross-Platform: The web-based interface works on any operating system through a browser, requiring no software installation on staff computers.
- Comprehensive Domain Coverage: All thirteen hostel management domains (from meal tracking to visitor logs) are modeled in the database.
- Open-Source Stack: PHP and MySQL are free and widely deployed, making the system deployable at no software licensing cost.

Limitations

- No Authentication: The system has no login page. Any user who can access the URL can perform all operations including deletes. This is a critical security gap for real-world deployment.
- SQL Injection Risk: The detail pages (student_details.php, etc.) pass raw GET parameters directly to SQL queries without intval() sanitization, leaving them vulnerable to SQL injection attacks.
- No Record Update: The system supports Add and Delete but not Edit/Update. Staff cannot correct a student's phone number or change a complaint's status through the UI.
- Manual ID Entry: Users must manually enter numeric IDs for records, which are error prone. An auto-increment suggestion or auto-populated ID field would improve usability.
- Partial Module Coverage: Nine of the thirteen database modules (allocation, transfer, invoice, payment, mess, entry log, leave, visitor, visitor log) are fully designed in the database but have no corresponding front-end interface.
- No Role-Based Access: All staff see and can modify all data. There is no separation between warden, mess manager, security guard, and admin roles.
- No Reporting: The system has no export, print, or report generation feature for financial summaries, occupancy rates, or complaint statistics.
- Unsecured Credentials: The database connection uses username root with an empty password, which is insecure for any environment beyond local development.

Future Improvements

- User Authentication: Implement a login system with hashed password storage (PHP password_hash() / password_verify()) and session management.
- Role-Based Access Control: Define permission levels for Admin, Warden, Mess Manager, and Security Guard, restricting access to relevant modules.
- Update (Edit) Functionality: Add edit forms for all entities so staff can correct records without deleting and re-adding them.
- Complete Module UI: Build front-end interfaces for room allocation, room transfer, invoice generation, payment recording, mess tracking, entry logs, leave requests, and visitor management.
- Dashboard Analytics: Add charts and graphs (using Chart.js or similar) showing occupancy rates, monthly revenue trends, and complaint resolution rates.
- PDF / Excel Reports: Integrate report generation for monthly invoices, occupancy lists, and payment summaries.
- Email Notifications: Trigger automated emails when complaints are resolved, invoices are generated, or leave requests are approved.
- Search Enhancement: Add name-based search (not just ID) and filters for status, department, and date ranges.
- Prepared Statements: Replace all mysqli_query() calls with prepared statements (PDO or mysqli_prepare()) to eliminate SQL injection risk.
- Responsive Mobile App: Develop a companion mobile application using a framework like React Native or Flutter.
- Audit Log: Record who made which change and when for accountability.

Conclusion

The Smart Hostel Management System successfully demonstrates the power of combining a well-designed relational database with a server-side web application to automate real-world administrative processes. The project covers all core concepts of a database systems course: entity-relationship modeling, schema normalization, SQL query formulation (including JOIN, INSERT, SELECT, and DELETE), referential integrity through foreign key constraints, and full-stack web development using PHP and MySQL.

The thirteen-table database schema covers every major operational domain of a university hostel — from student admissions and room management to financial invoicing, meal tracking, gate entry logs, visitor management, and complaint resolution. Although not all modules have front-end interfaces in this version, their complete database designs serve as a blueprint for future improvements.

The front-end interface provides a modern and user-friendly design through carefully planned CSS styling with dark theme, teal accent colors, responsive grid layouts, and interactive JavaScript components. The dashboard provides quick access into hostel operations through live database-based statistics.

This project also highlights the importance of database design decisions — such as choosing appropriate data types, enforcing constraints, and normalizing to 3NF — in building reliable, maintainable software systems. It also points out areas that can be improved such as security (authentication, prepared statements) and completeness (update operations, full module interfaces), which will guide the next phase of development.

References

32. PHP Manual. (2024). MySQLi — MySQL Improved Extension. https://www.php.net/manual/en/book.mysqli.php
33. MySQL Documentation. (2024). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/
34. MariaDB Documentation. (2024). MariaDB 10.4 Server Documentation. https://mariadb.com/kb/en/documentation/
35. W3Schools. (2024). PHP Tutorial. https://www.w3schools.com/php/
36. W3Schools. (2024). SQL Tutorial. https://www.w3schools.com/sql/
37. MDN Web Docs. (2024). HTML5 Reference. https://developer.mozilla.org/en-US/docs/Web/HTML
38. MDN Web Docs. (2024). CSS Reference. https://developer.mozilla.org/en-US/docs/Web/CSS
39. Google Fonts. (2024). Bebas Neue and DM Sans Font Families. https://fonts.google.com/
40. phpMyAdmin Documentation. (2024). phpMyAdmin 5.x Documentation. https://www.phpmyadmin.net/docs/
41. Elmasri, R. & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.
42. Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377-387.

Short Presentation Summary

Use the following bullet points as a concise slide-by-slide presentation guide:

Slide 1 — Title

- Smart Hostel Management System — Database Systems PBL Project
- Team Members | Instructor | Date

Slide 2 — Problem

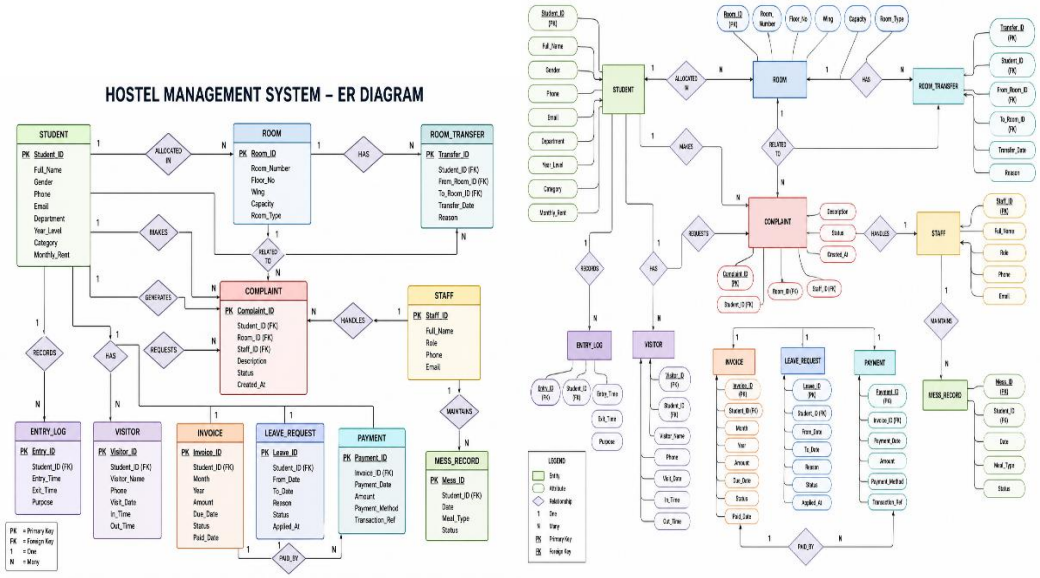
- Manual hostel records are slow, error-prone, and scattered.
- No quick access into room occupancy, complaints, or payments.

Slide 3 — Our Solution

- Web-based administration dashboard powered by PHP + MySQL.
- 13-table normalized database covering all hostel operations.

Slide 4 — Database Schema

- 13 entities: student, room, staff, complaint, allocation, transfer, invoice, payment, mess, entry, leave, visitor, visitor_log.
- All connected via foreign keys. InnoDB engine. 3NF normalized.



Slide 5 — Technology Stack

- Backend: PHP 8.0 + MySQL (MariaDB 10.4)
- Frontend: HTML5, CSS3 (custom), JavaScript (vanilla)
- Tools: XAMPP, phpMyAdmin

Slide 6 — Key Features

- Live dashboard stats, student/room/staff/complaint CRUD
- Multi-table JOIN search, cascading deletes, status badges

Slide 7 — Live Demo

- [Show index.php running on localhost]
- Add a student, search it, view detail, delete it

Slide 8 — Limitations & Future Work

- No login yet — authentication to be added
- 9 DB modules need UI — invoice, mess, entry log, visitor

Slide 9 — Conclusion

- Demonstrates complete DB design + full-stack development
- Ready to scale with authentication and reporting features